

Internal state in evolved visual control: Reactive and proactive controllers for Chromium dino

AE4350: Bio-inspired Intelligence and Learning for Aerospace Applications

Name Student number

Jop de Vries 5364477

Date: 30 August 2026

Delft University of Technology, Faculty of Aerospace Engineering

Abstract

This study tests whether persistent internal state improves evolved visual control in the Chromium Dino task. Parameter-count-matched feed-forward and recurrent controllers received the same five visual inputs and 2500 candidate evaluations per run. The feed-forward controllers reached the 3600-step training ceiling in all three runs; the recurrent controllers did not. Across three matched runs evaluated on five held-out worlds, neither architecture was consistently better. The mean architecture ordering also changed across the tested initialization-and-mutation scales and parent-offspring configurations. Holding the first normalized five-feature observation vector constant reduced both selected policies to about 271 steps, showing dependence on updated visual input. Resetting recurrent state before every control step reduced the recurrent representative's mean survival from 2683 to 271 steps under the same genome, world seeds, and sensing pipeline. On 20 locked test worlds, the selected feed-forward controller averaged 3379 steps and outperformed the recurrent controller, which averaged 1625 steps, on 19 worlds. The selected recurrent controller depended on persistent recurrent state, but this did not translate into better survival in validation or in the locked test.

1 Introduction

Evolutionary Robotics (ER) develops controllers by testing candidates in an environment and scoring them on the task; selection and variation then produce the next population. The resulting controllers are shaped by the closed-loop interaction between their actions and subsequent observations.

This project applies ER to a Chromium-derived version of the dino game. The dino avoids cacti and pterodactyls by jumping or ducking as the game gradually speeds up. The controller receives no game-state variables: its input consists of five geometric features extracted from the current rendered frame, and it chooses among no key, jump, and duck/fast-drop.

The five current-frame measurements do not uniquely determine the underlying dynamic state. The same vertical dino position may occur during ascent and descent, and world speed is absent from the controller input. Different histories can therefore produce similar observations. A reactive controller acts only on the current observation, whereas a proactive controller may also use information stored in its internal state. Whether the evolved proactive controller uses that state must therefore be tested.

The experiments compare parameter-count-matched reactive and proactive controllers; both use the same inputs, actions, environment, fitness criterion, and evaluation budget. The analysis covers task performance, learning curves, held-out environments, and evolved sensorimotor strategies.

The main research question is:

How do reactive and proactive evolved controllers differ in learning, generalisation, and sensorimotor behaviour in the Chromium dino task?

The subquestions are:

1. How do reactive and proactive controllers differ in learning and held-out survival?
2. How sensitive is this difference to the configured initialization-and-mutation scale and the parent-offspring population configuration?
3. How do their evolved strategies differ, and does the proactive controller functionally use its internal state?

2 Theoretical background

2.1 Evolutionary robotics and sensorimotor coordination

In this project, controller parameters are evolved directly: a genome defines a controller, its behaviour in the environment determines its fitness, and selection and variation produce new candidates,

$$\begin{aligned} \text{genome} &\rightarrow \text{controller} \rightarrow \text{behaviour} \\ &\rightarrow \text{fitness} \rightarrow \text{selection and variation.} \end{aligned} \quad (2.1)$$

The controller is therefore judged by what its closed-loop behaviour achieves, rather than against prescribed actions for individual sensory inputs.

This evolutionary process is closely related to sensory-motor coordination. A situated agent acts in the same environment that it senses,

$$\vec{i}_t \rightarrow \vec{a}_t \rightarrow \vec{x}_{t+1} \rightarrow \vec{i}_{t+1}, \quad (2.2)$$

where $\vec{i}_t$ is the sensory input, $\vec{a}_t$ the action, and $\vec{x}_t$ the agent–environment state. An action changes that state and therefore changes what is sensed next. Nolfi describes sensory–motor coordination as exploiting the agent–environment interaction, specifically the ability to select favourable sensory patterns through motor actions [2, p. 120]. A reactive controller can therefore exploit temporal structure through this external feedback loop without maintaining persistent internal state.

2.2 Perceptual ambiguity and internal state

Nolfi characterises reactive systems as producing the same motor action for the same sensory state [2, p. 120]. For the present comparison, this is represented as

$$\vec{a}_t = f(\vec{i}_t). \quad (2.3)$$

Its current sensory input therefore fixes its immediate action. Nolfi defines sensory aliasing as the case in which two or more agent–environment states correspond to the same sensory pattern but require different motor responses [2, p. 121]. In the dino task, the same vertical position can occur while rising or falling, and the same obstacle separation can occur at different game speeds. These examples establish that the current observation may correspond to different underlying dynamic states; whether those states require different actions is an empirical question.

Nolfi notes that an agent can act to search for sensory patterns that are not affected by aliasing [2, p. 121]. Such actions also partially determine the sensory patterns experienced next [2, p. 122]. By construction, the reactive controller used here has no persistent internal

state in which to retain information unavailable from the current input.

A proactive controller adds an internal state,

$$\vec{s}_{t+1} = g(\vec{i}_t, \vec{s}_t), \quad \vec{a}_t = f(\vec{i}_t, \vec{s}_{t+1}). \quad (2.4)$$

The current action can then depend on the present sensory input and on information retained from earlier interaction. According to Beer, an agent with internal state can respond differently to the same environmental situation depending on that state; the state, in turn, can depend on past experiences [1, p. 238].

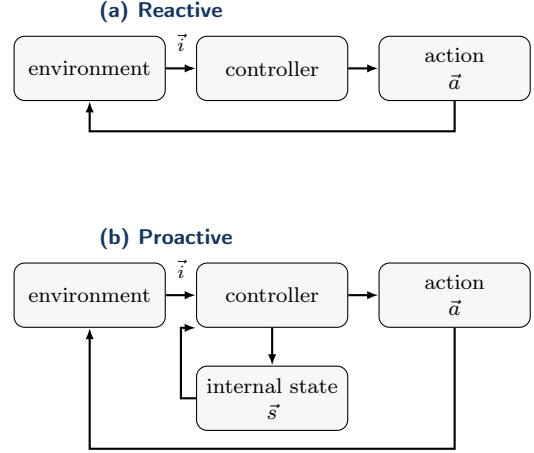


Figure 1: Both controllers exploit the sensorimotor loop through the environment. Only the proactive controller retains internal state between control steps.

Internal state therefore provides an additional way to use information from the preceding trajectory, but its presence does not prove that evolution uses it. The experiments must distinguish improved performance from actual functional dependence on recurrent state.

3 Method

3.1 Environment and visual interface

The experiment uses a Chromium-derived port at the following fixed revision:

Commit: 1ccb91e11f09fbbdec4f8f754d0e2f7d28246660
Source: components/neterror/resources/dino_game

The port retains the normal-mode geometry, collision rules, jump and duck mechanics, obstacle generation and speed progression. Seeded randomness and a fixed 60 Hz step make runs repeatable.

Candidates share world time, game speed and obstacle sequence; each candidate has its own dino physics, actions, rendered frames and controller state. Thus, they experience the same external world without interacting with or observing one another.

Figure 2 shows the boundary between rendered pixels and controller actions.

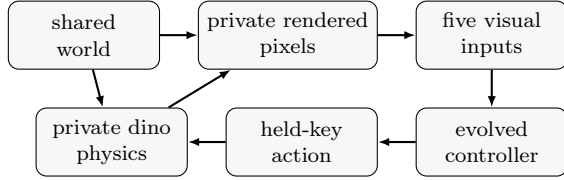


Figure 2: Experimental control loop for one candidate. All candidates share the obstacle world, but perception, controller state, actions and dino dynamics are private. No engine state crosses this boundary.

Perception receives lossless pixel data equivalent to each clean 600×150 rendered frame. Neither game speed nor dino velocity is provided, and the input also excludes obstacle objects, earlier observations and actions, optical flow and time-to-collision. At each step, a fixed perception front-end segments these pixels and derives five measurements from the bounding boxes of connected components.

Figure 3 defines the measurements in the rendered frame. Image coordinates start at the top-left; x increases rightwards and y downwards.

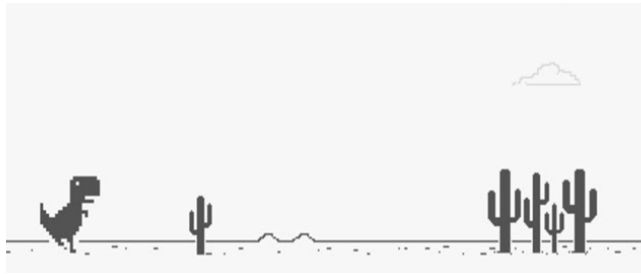
Table 1: Visual controller inputs before normalisation.

Input	Pixel measurement	Clip [px]
i_1	nearest obstacle left edge minus dino right edge	$[-100, 600]$
i_2	nearest obstacle top edge minus dino top edge	$[-150, 150]$
i_3	obstacle bounding-box width	$[0, 100]$
i_4	obstacle bounding-box height	$[0, 70]$
i_5	dino bounding-box top coordinate	$[0, 150]$

Clipping and normalisation map each value to $[0, 1]$:

$$\hat{x} = \frac{\text{clip}(x, l, u) - l}{u - l}. \quad (3.1)$$

(a) Rendered game frame



(b) Measurements extracted from the frame

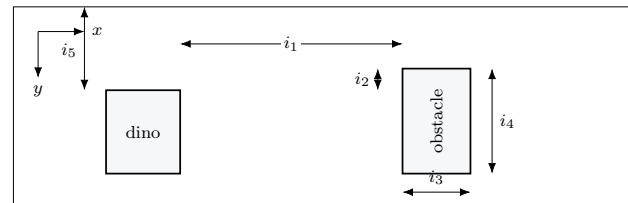


Figure 3: Chromium dino task and the five geometric measurements extracted from the current frame. Panel (b) is schematic; image coordinates originate at the top-left.

Every control step starts with fresh rendered pixels, so the perception system carries neither history nor a temporal derivative. An unavailable measurement is logged and its normalized channel value is set to zero; normalized zero therefore represents either an unavailable measurement or the channel’s normalized lower bound. Hidden game state is never used as a replacement.

The controller selects one of three key states:

Table 2: Controller action space.

Action	Key state	Effect
0	no key	run / release key
1	jump held	start/continue jump
2	duck held	duck on ground / fast-drop in air

The largest score determines the action; the first index resolves exact ties. A jump begins only when the key changes from released to held, and holding it alters the trajectory. If the dino lands while the key remains held, a second jump cannot begin. Press and release timing form part of the control problem.

Chromium’s speed parameter begins at 6, rises by 0.001 per frame and is capped at 13. Obstacle generation begins just after 3s of logical game time. Available obstacles are small cacti, large cacti and pterodactyls. Pterodactyls enter the pool once the speed parameter reaches 8.5. These game parameters are hidden from the controller and should not be read as physical velocities.

3.2 Controllers

Both networks map five inputs to three action scores and contain 93 evolved weights and biases. Equal genome lengths remove parameter count as a confound, although they do not equate the architectures’ dynamics or ease of optimisation.

The reactive feed-forward neural network (FFNN) uses a $5 \rightarrow 10 \rightarrow 3$ architecture:

$$h_t = \tanh(W_{\text{in}} i_t + b_h), \quad z_t = W_{\text{out}} h_t + b_o. \quad (3.2)$$

Its hidden activation is discarded after each step.

The proactive continuous-time recurrent neural network (CTRNN) uses a $5 \rightarrow 6_{\text{rec}} \rightarrow 3$ architecture:

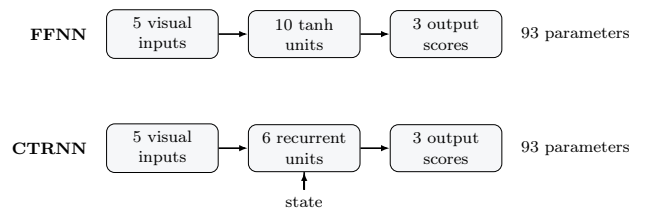


Figure 4: Controllers matched by parameter count. The FFNN recomputes its hidden activation each step; the CTRNN carries six recurrent states between steps.

$$\begin{aligned}
h_{t+1} &= h_t + \frac{\Delta t}{\tau} [-h_t + \tanh(W_{\text{in}} i_t + W_{\text{rec}} h_t + b_h)], \\
z_t &= W_{\text{out}} h_{t+1} + b_o.
\end{aligned}
\tag{3.3}$$

The integration step is $\Delta t = 1/60$ s, the time constant is $\tau = 0.25$ s, and $h_0 = 0$; the six recurrent states are reset before each episode so that no information is transferred between world seeds. The time constant was fixed before analysis and is shared across experiments.

Each genome is a contiguous `float64` vector decoded in row-major order. The FFNN stores $W_{\text{in}}, b_h, W_{\text{out}}, b_o$; the CTRNN also stores W_{rec} .

3.3 Evolutionary optimisation

Fitness is mean survival over the ten training seeds, measured in 60 Hz control steps. An episode ends at game over or after 3600 steps (60 s). No obstacle bonus, action penalty or secondary objective modifies this value. Every candidate faces the same ten worlds. A fixed 270-step no-action reference is shown in the result figures for context; no separate no-action replay is included in the analysed dataset. Scores of 3600 are capped observations: they show that the controller survived the full episode, not when it would otherwise have failed.

The elitist (20 + 80)-Evolution Strategy retains 20 parents and generates 80 offspring per generation:

For RQ1, the initial 20 parents are independent samples from a zero-centred Gaussian with standard deviation 0.3. In the RQ2 scale conditions, the configured σ is used both for initial population sampling,

$$\theta_0 \sim \mathcal{N}(0, \sigma^2 I),$$

and as the fixed mutation scale for subsequent offspring. The RQ2 scale comparison therefore changes initialization scale and mutation magnitude jointly. Each offspring perturbs one source parent:

$$\theta' = \theta_p + \sigma_g \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \tag{3.4}$$

The parameter σ_g sets the typical mutation magnitude.

Fitness is unavailable before the first evaluation, so each parent receives four offspring. Later generations assign one offspring to every retained parent; the remaining

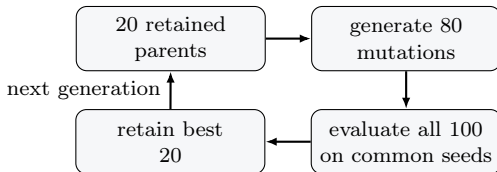


Figure 5: One evolutionary generation. The 20 parents and 80 mutations are evaluated together; the best 20 form the next parent population.

offspring source parents are sampled with replacement using probabilities proportional to linear fitness-rank weights. Rank weights decrease from 20 for the best parent to 1 for the worst, with tied parents receiving the same mean rank. The standard 20 + 80 setting therefore has 20 guaranteed and 60 rank-sampled offspring.

The candidate pool includes both parents and offspring. A stable descending sort retains the best 20 of all 100 candidates; because parent entries precede offspring, an exact tie preserves the incumbent.

RQ1 adapts mutation scale from the fraction of offspring that strictly outperform their source parent:

Table 3: RQ1 mutation-scale adaptation. Ordinary updates are clipped to $0.03 \leq \sigma_g \leq 0.30$; the plateau rule applies when all retained parents tie and no offspring improves its source parent.

Outcome	Next scale	Purpose
< 20% improve	$0.85\sigma_g$	search more locally
20% improve	unchanged	target success rate
> 20% improve	$\sigma_g/0.85$	explore farther
complete plateau	0.60	escape a locally flat region

The adaptation scheme was a one-fifth-success-rule-inspired heuristic; the plateau reset to $\sigma_g = 0.60$ was an experiment-specific escape mechanism fixed before analysis.

A plateau sets the next generation’s scale to 0.60. Once fitness is no longer flat, bounded adaptation resumes.

3.4 Experimental design

A complete evolutionary run is the experimental unit. Each condition uses optimiser seeds 7, 17 and 27; architectures are paired by optimiser-seed label and common initial random stream.

The environment seeds are fixed before analysis:

Table 4: World-seed partitions. Fitness uses the training set only.

Set	World seeds
Training	101, 202, 707, 808, 909, 1001, 1102, 1203, 1304, 1405
Validation	303, 404, 1303, 1404, 2303
Locked test	505, 606, 1505, 1606, 2505, 2606, 3505, 3606, 4505, 4606, 5505, 5606, 6505, 6606, 7505, 7606, 8505, 8606, 9505, 9606

Evolution uses only the training seeds. The locked test set is not evaluated until optimisation, sensitivity analysis and model selection are complete.

Table 5: Experimental design. RQ2 changes one factor relative to the $20 + 80, \sigma = 0.3$ reference condition.

	Experiment	Purpose
RQ1	FFNN versus CTRNN using adaptive $20 + 80$ evolution	Compare learning and held-out survival under equal budgets.
RQ2	Fixed $\sigma = 0.1, 0.3, 0.6$ at $20 + 80$; and $10 + 90, 20 + 80, 40 + 60$ at fixed $\sigma = 0.3$	Measure sensitivity to the combined initialization-and-mutation scale and to the parent-offspring population configuration.
RQ3	Normal replay, interventions on visual inputs and CTRNN state reset	Test which sensory information and recurrent dynamics are functionally used.

A standard run evaluates 100 candidates over 25 generations: 2500 candidate evaluations and 25 000 episodes on training seeds. Three seeds per architecture give six RQ1 runs.

RQ2 disables mutation adaptation and plateau escape. Its five unique conditions are

$$\begin{aligned}
 &20 + 80, \sigma = 0.1, \quad 20 + 80, \sigma = 0.3, \\
 &20 + 80, \sigma = 0.6, \quad 10 + 90, \sigma = 0.3, \\
 &40 + 60, \sigma = 0.3.
 \end{aligned} \tag{3.5}$$

Testing each condition with both architectures and three optimiser seeds gives $5 \times 2 \times 3 = 30$ RQ2 runs. Population size, number of generations and total candidate budget are equal across conditions, and runs with $\sigma = 0.3$ are conducted separately from adaptive RQ1 runs.

In the scale comparison, the configured σ controls both the spread of the initial population and the magnitude of subsequent mutations; these effects are therefore not separated.

RQ3 replays each frozen representative on the five validation seeds under normal input, a constant first normalized five-feature observation vector, and each zeroed channel. A separate CTRNN replay resets recurrent state to zero before each step while the genome, world seed and observation-generation pipeline remain unchanged. Subsequent actions, trajectories and observation values may diverge. Interventions are paired with normal replay by world seed. For each world, action disagreement is the fraction of paired decisions that differ over the shared trajectory up to the earlier termination step; reported disagreement is the arithmetic mean of these equally weighted per-world fractions.

Within each architecture, the RQ1 archive with median validation performance (reactive seed 17 and proactive seed 27) is frozen for RQ3 and the locked final test. After all analyses and selection are complete, each selected controller is evaluated once on all 20 locked-test

seeds.

RQ1 and RQ2 report three independent runs, arithmetic means and two-sided 95% Student- t intervals; architectures are paired by optimiser seed. With $n = 3$, the intervals describe between-run variation and provide limited power for significance testing.

Across five RQ3 validation worlds and 20 locked-test worlds, variation describes environments for the frozen representatives, not independently evolved controllers; ceiling fractions are reported where relevant.

4 Results

4.1 RQ1: Learning and held-out survival

RQ1 asks which architecture learned faster within the common 2500-candidate budget and whether either architecture was consistently better on held-out worlds. Figure 6 addresses the first question by plotting selected-parent training survival for three independent optimiser runs per architecture.

Both architectures begin near the fixed 270-step no-action reference, but the reactive curves accelerate much earlier. All three reactive runs reach the 3600-step ceiling within the budget, whereas all three proactive runs remain below it and are still rising at the final generation. Figure 7 addresses the separate held-out question: Panel A shows validation survival for matched optimiser runs, while Panel B shows the direct proactive-minus-reactive effect.

The paired effect is positive for seed 7 but negative for seeds 17 and 27, and its two-sided 95% Student- t interval is large relative to the mean difference. The reactive architecture learned substantially faster within the available budget, but held-out validation did not establish a consistent architecture-level advantage.

4.2 RQ2: Parameter sensitivity

RQ2 tests whether the RQ1 architecture comparison changes with the configured initialization-and-mutation scale or the parent-offspring population configuration. In Figure 8, Panels A and C vary the common initialization-and-mutation scale at fixed $20 + 80$, while Panels B and D vary the parent-offspring population configuration at fixed $\sigma = 0.3$. The upper panels show each architecture separately; the lower panels show the three paired seed effects and their uncertainty.

At fixed $20 + 80$, the reactive mean changes comparatively little, whereas the proactive mean is below the reactive mean at $\sigma = 0.1$ and above it at the two larger configured scales. At fixed $\sigma = 0.3$, the reactive mean is similar under $10 + 90$ and lower under $40 + 60$, whereas the proactive mean falls sharply at both alternatives to $20 + 80$. The lower panels show wide paired intervals, all of which include zero. Because the design is one-factor-

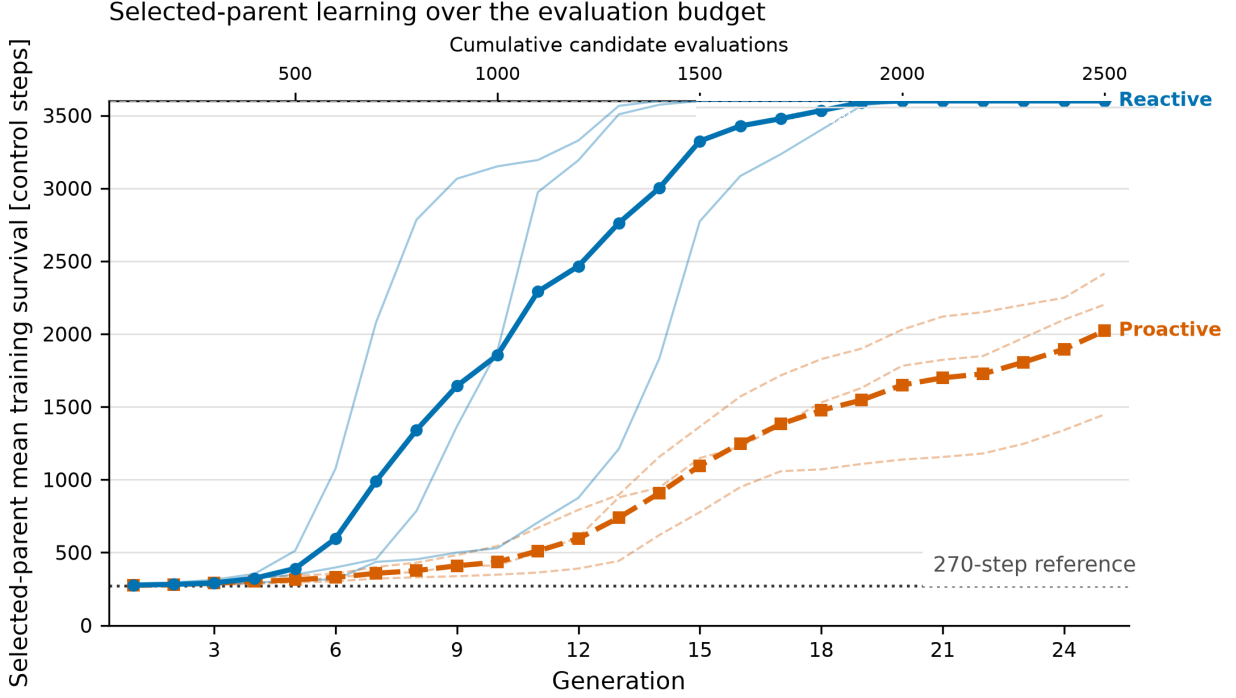


Figure 6: Selected-parent training survival over the 2500-candidate RQ1 budget. Thin lines show mean survival of the 20 retained parents within each independent optimiser run ($n = 3$ per architecture); thick lines show the arithmetic mean across the three runs, and dotted lines mark the fixed 270-step no-action reference and the 3600-step ceiling.

at-a-time, interactions between the configured scale and the parent-offspring population configuration were not estimated. The mean architecture ordering changed across both tested factors, and no tested condition produced a paired interval that excluded zero.

4.3 RQ3: Sensorimotor strategy and internal state

After comparing performance, RQ3 asks what the frozen representatives did, whether their policies depended on changing visual feedback, and whether the proactive policy used persistent recurrent state. Figure 9 organises the behavioural and intervention evidence in that order.

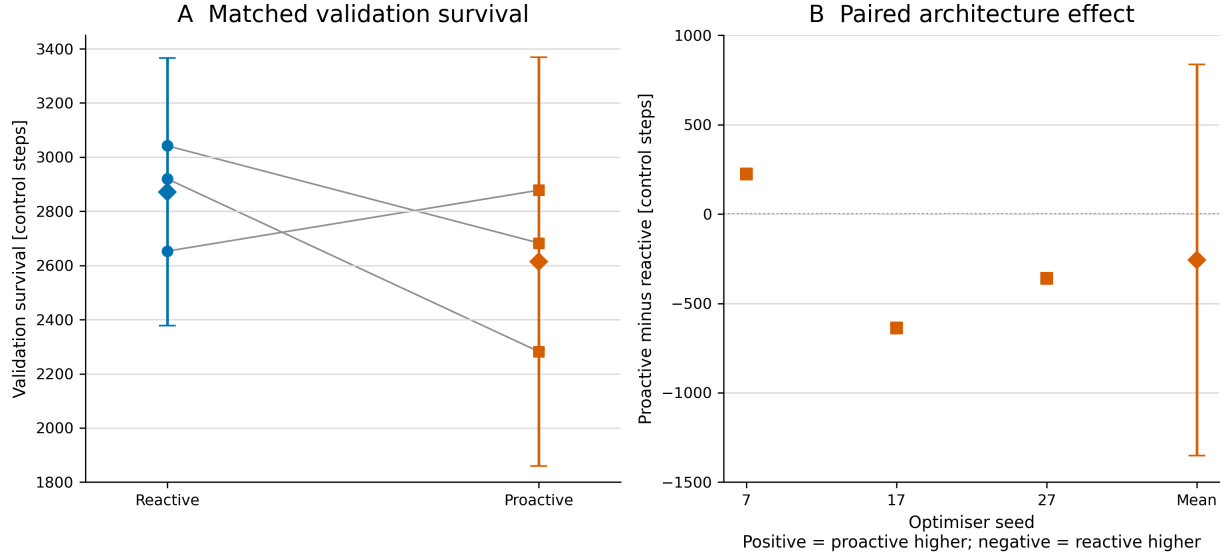


Figure 7: Mean survival on five validation worlds for $n = 3$ independent optimiser runs per architecture, paired by seed. Thin lines connect matched runs; diamonds and error bars show means with two-sided 95% Student- t intervals; positive values in Panel B favour proactive.

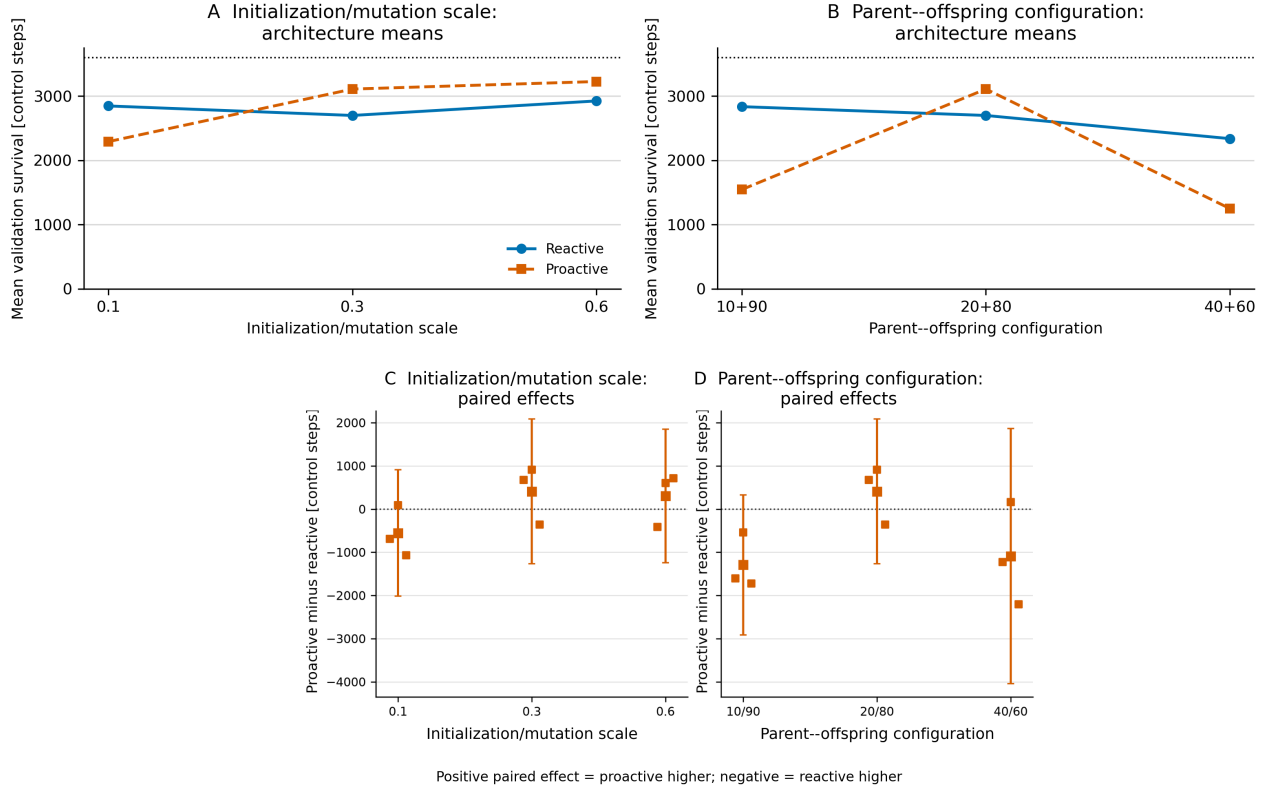


Figure 8: Validation sensitivity across the five one-factor-at-a-time RQ2 conditions ($n = 3$ independent optimiser runs per architecture). The left panels vary the shared initialization/mutation scale at fixed $20 + 80$; the right panels vary the parent-offspring population configuration at fixed $\sigma = 0.3$. Panels A–B show architecture means; panels C–D show paired seed effects and their means with two-sided 95% Student- t intervals, where positive values favour proactive.

Overt strategy. Both representatives evolved a jump/release avoidance strategy and never selected duck/fast-drop, despite encountering pterodactyls in all five replay worlds. Panel A is a coarse description of action use and does not show when either policy initiated or released a jump.

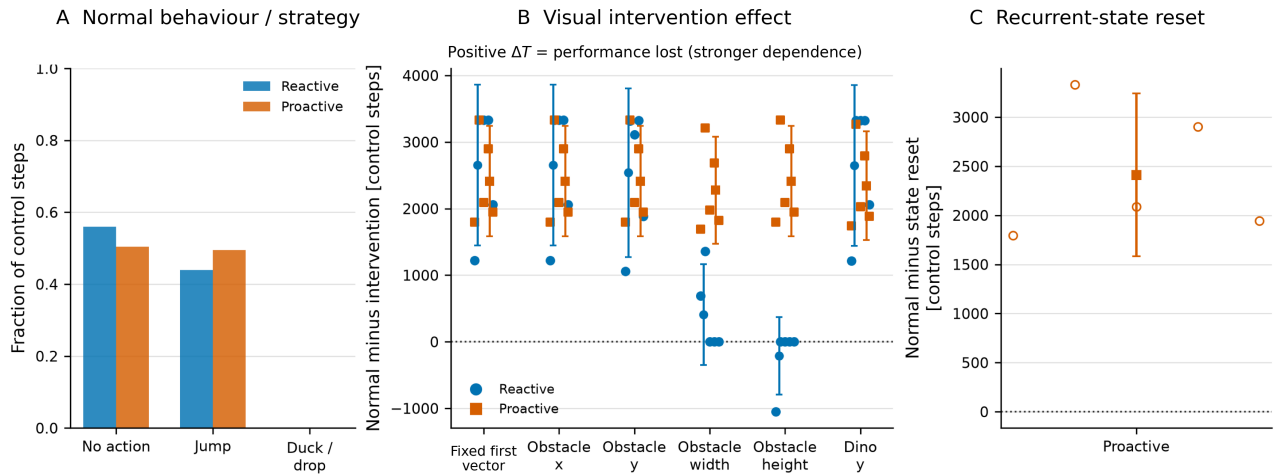


Figure 9: Normal behaviour and intervention effects for one frozen representative per architecture over five matched validation worlds. Panel A shows action occupancy; panels B–C show $\Delta T = T_{\text{normal}} - T_{\text{intervention}}$, with small markers for worlds and large markers plus error bars for the mean and two-sided 95% Student- t interval, so positive values denote performance lost under the intervention.

Visual-feedback dependence. Holding the first five-feature observation vector constant removes subsequent visual updating at the controller input while leaving the controller active. Under this intervention, survival for both policies falls to approximately the fixed 270-step no-action reference, and both continue to select jump held. Both policies therefore functionally depend on time-varying visual feedback. The single-channel interventions show large effects for obstacle-relative x , obstacle-relative y , and dino y in the reactive policy, but much smaller effects for obstacle width and height; all five zeroed channels produce large survival losses for the proactive policy. For obstacle-relative y , the large reactive survival loss coincides with only 2.2% mean per-world action disagreement. Zeroing a channel is a strong counterfactual, not an estimate of independent feature importance.

Recurrent-state dependence. Panel C compares the same frozen genome in the same worlds with the same observation-generation pipeline. Persistent recurrent state is removed by resetting $h = 0$ before every control step; changed actions may then alter the dino trajectory and later observations. Mean survival falls from 2683 to 271 steps, with a decrease in all five matched worlds. Mean per-world action disagreement over the shared pre-termination trajectories is 20.7%. The proactive representative therefore functionally depended on persistent recurrent state, although the intervention does not identify what that state represented.

The representatives used similar overt jump/release behaviour, but differed in information dependence: both required changing visual feedback, while the proactive policy additionally required persistent recurrent state.

4.4 Locked final test

After optimisation and representative selection were frozen, the two selected policies were evaluated on 20 previously unused worlds.

Table 6: Locked-test survival for one frozen representative per architecture on $n = 20$ matched previously unused worlds. Two-sided 95% Student- t intervals describe environment-seed variation, not independent evolutionary replication.

Controller	Mean $\pm$ 95% CI	Median	Ceiling
Reactive	3379 $\pm$ 286	3600	17/20
Proactive	1625 $\pm$ 412	1562	0/20

The reactive representative averaged 3379 steps with 17 of 20 episodes at the ceiling; the proactive representative averaged 1625 steps and never reached the ceiling. Reactive survival was higher on 19 of the 20 matched worlds.

5 Discussion

Learning and generalisation. RQ1 shows a clear difference in finite-budget learning: the reactive architecture improved substantially faster and reached the evaluation ceiling, whereas all proactive runs were still improving when the budget ended. This does not establish a lower attainable performance for the CTRNN. Equal parameter count does not imply equal optimisation difficulty either. In the FFNN, a parameter change modifies a static mapping from the current observation to action scores. In the CTRNN, however, the same type of mutation can also alter the recurrent trajectory and thus affect later actions, internal states and observations. The slower proactive learning may therefore reflect a more temporally coupled search problem, although recurrence cannot be isolated as its cause because the architectures also differ in hidden width, connectivity and dynamics. Held-out validation provides no consistent architecture-level generalisation advantage: the paired effect changes sign across optimiser seeds. The locked test supports the narrower conclusion that the selected reactive policy generalised better, surviving longer in 19 of 20 worlds. These worlds characterise the two frozen policies rather than independent evolutionary replication.

Optimiser sensitivity. RQ2 further shows that the architecture comparison depends on the evolutionary-search configuration. In the scale experiment, the configured σ jointly determined the spread of the initial population and the magnitude of subsequent mutations. The comparison therefore measures sensitivity to this combined scale parameter rather than isolating mutation magnitude. The parent-offspring configuration changes the number of retained elites and generated offspring and, through the rank-based offspring allocation, also changes the resulting selection pressure under the fixed population budget. Across the tested initialization/mutation scales, the reactive validation mean varied less than the proactive mean. The proactive mean was also highest at the intermediate 20 + 80 configuration and lower at both tested alternatives. Consequently, the mean architecture ordering reversed across both factors. This pattern in the observed means is consistent with greater sensitivity of the recurrent controller to the tested evolutionary-search configurations, but the OFAT design, the coupled initialization/mutation scale, and the architectural differences prevent attributing this specifically to recurrence or to mutation magnitude alone. The RQ1 comparison should therefore be interpreted as conditional on the evolutionary search configuration rather than as an optimiser-independent ranking of the two architectures.

Evolved sensorimotor mechanisms. RQ3 distinguishes similar overt behaviour from different underlying

information dependence. Both representatives evolved a jump/release strategy and never used duck/fast-drop, despite encountering pterodactyls. The reactive ablations show more specifically which inputs supported this solution: survival depended strongly on obstacle-relative position and dino vertical position, but much less on obstacle width and height. The selected reactive policy was substantially more sensitive to interventions on obstacle-relative position and dino vertical position than to interventions on obstacle width and height. This does not imply that it reconstructed unobserved quantities such as velocity or jump phase. Instead, the constant-input intervention shows that the policy requires continuously updated visual feedback. A reactive controller can therefore exploit temporal consequences externally: an action changes the dino trajectory, which changes the next observation on which the next action is based, as described by the sensorimotor loop in Section 2.

The obstacle-relative- y intervention illustrates why such closed-loop behaviour cannot be interpreted through linear feature importance. A large survival loss occurred despite only 2.2% mean per-world action disagreement before the earlier termination. A small initial action difference can alter the subsequent trajectory; observations and later actions can then diverge. The intervention effect must therefore be interpreted at trajectory level. The selected proactive policy used a second temporal mechanism inside the controller. Resetting its recurrent state while preserving the genome, world seed and observation-generation pipeline caused survival to collapse and changed the action sequence. The resulting trajectories and later observations could then diverge in closed loop. This directly demonstrates functional dependence on persistent state. The architecture-level experiments nevertheless show no consistent performance benefit from that additional mechanism. State capacity, functional state use and performance benefit are therefore distinct: the selected CTRNN policy functionally depended on its state, but the architecture-level experiments showed no consistent learning or held-out advantage under the tested conditions. The interventions provide no evidence for assigning speed, vertical velocity, phase or another specific latent quantity to the recurrent state.

Limitations. The experimental design places several limits on these conclusions. The fixed 2500-candidate budget establishes finite-budget learning speed, but the still-improving proactive runs leave their attainable performance unresolved. Only three independent optimiser runs were available per RQ1/RQ2 condition, so the reported intervals describe substantial between-run uncertainty and do not support strong significance claims. The 3600-step horizon right-censors high-performing re-

active episodes, preventing differences above that level from being resolved. RQ3 and the locked test use one validation-frozen representative per architecture; variation across their five and 20 worlds therefore describes those policies rather than variability across independently evolved controllers. Parameter-count matching removes genome dimensionality as one confound, but does not isolate recurrence as the sole architectural difference. In the RQ2 scale experiment, the configured σ was used for both initial population sampling and offspring mutation, so initialization-scale and mutation-scale effects cannot be separated. RQ2 is one-factor-at-a-time, so interactions between the configured scale and the parent-offspring population configuration were not estimated. Finally, the conclusions apply to the chosen five-feature visual abstraction, and zero-channel interventions are strong counterfactual tests of functional dependence rather than independent estimates of feature importance.

6 Conclusion

The reactive architecture learned substantially faster within the fixed evaluation budget. Validation, however, did not establish a consistent architecture-level advantage, and the mean ordering changed across the tested optimiser settings. On the locked worlds, the selected reactive policy nevertheless substantially outperformed the selected proactive policy.

Both selected policies used jump/release behaviour and required time-varying visual feedback. The reactive policy succeeded without persistent controller state, consistent with coordination through current visual geometry and the external sensorimotor loop; the proactive policy also depended on persistent recurrent state. The policies therefore differed in mechanism, but this difference did not translate into a consistent performance advantage across the tested conditions.

Code availability. The project source code is available at github.com/Jopdevries/dino-er.

Declaration of AI use

ChatGPT was used as a supporting tool for language refinement, L^AT_EX formatting, drafting and refining parts of the project README, and selected software-engineering tasks. Programming assistance was limited to debugging, writing tests, developing the graphical user interface, and setting up the uv development environment. GitHub Copilot was used for minor code completion and syntax suggestions.

The modelling approach, controller design, experimental design, and implementation of the models, controllers, and experimental pipeline were developed by the author. AI tools were not used to perform the data analysis or to determine the interpretation of the results and scientific conclusions. They were not used as a substitute for understanding the theoretical principles or technical methods applied in this work.

All AI-assisted code and text were critically reviewed, tested where applicable, and adapted before inclusion. The author takes full responsibility for the submitted report and code, including the correctness of the models and implementation, the experimental methodology, the reported results, their interpretation, and the conclusions.

References

- [1] Randall D. Beer. The dynamics of active categorical perception in an evolved model agent. *Adaptive Behavior*, 11(4):209–243, 2003.
- [2] Stefano Nolfi. Power and limits of reactive agents. *Neurocomputing*, 42:119–145, 2002.